

Testing APIs with Postman

 [. \(https://github.com/learn-co-curriculum/python-p4-getting-remote-data-postman\)](https://github.com/learn-co-curriculum/python-p4-getting-remote-data-postman)  [. \(https://github.com/learn-co-curriculum/python-p4-getting-remote-data-postman/issues/new\)](https://github.com/learn-co-curriculum/python-p4-getting-remote-data-postman/issues/new)

Learning Goals

- Use different kinds of clients to make requests.
 - Compose a request with a URL, HTTP verb, headers, and body.
 - Test non-GET requests using Postman.
-

Key Vocab

- **Request:** an attempt by one machine to contact another over the internet.
 - **Client:** an application or machine that accesses services being provided by a server through the internet.
 - **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
-

Video Walkthrough

Curriculum - Phase 4 - Testing APIs With Postman



HTTP Clients

When developing a web application with Flask, our primary goal is to take **HTTP requests** and return an appropriate **response**, following the idea of **client-server** communication. So far, we've been using one particular client to interact with our server: the browser!

It's straightforward to use the browser to make a GET request — all we have to do is enter an address in the browser's URL bar, such as `http://localhost:9292/games` , and hit enter.

However, we also have learned a little about other HTTP verbs aside from `GET` , such as `POST` , `PATCH` , and `DELETE` . Using these other HTTP verbs is a strong convention when working with APIs in particular: they signal to developers that the server is going to perform a specific kind of action, based on which HTTP verb is used in the request:

- **POST** (Create): create a new resource in a database
- **GET** (Read): access/query information from a database
- **PATCH** (Update): update one specific resource in a database
- **DELETE** (Delete): delete one specific resource from a database

We've seen how to make a POST request in JavaScript using fetch:

```
fetch("http://localhost:9292/games", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({  
    title: "Breath of the Wild",  
    platform: "Switch",  
  }),  
});
```

However, running a JavaScript application and writing `fetch` requests to test out our API in development would be quite a chore.

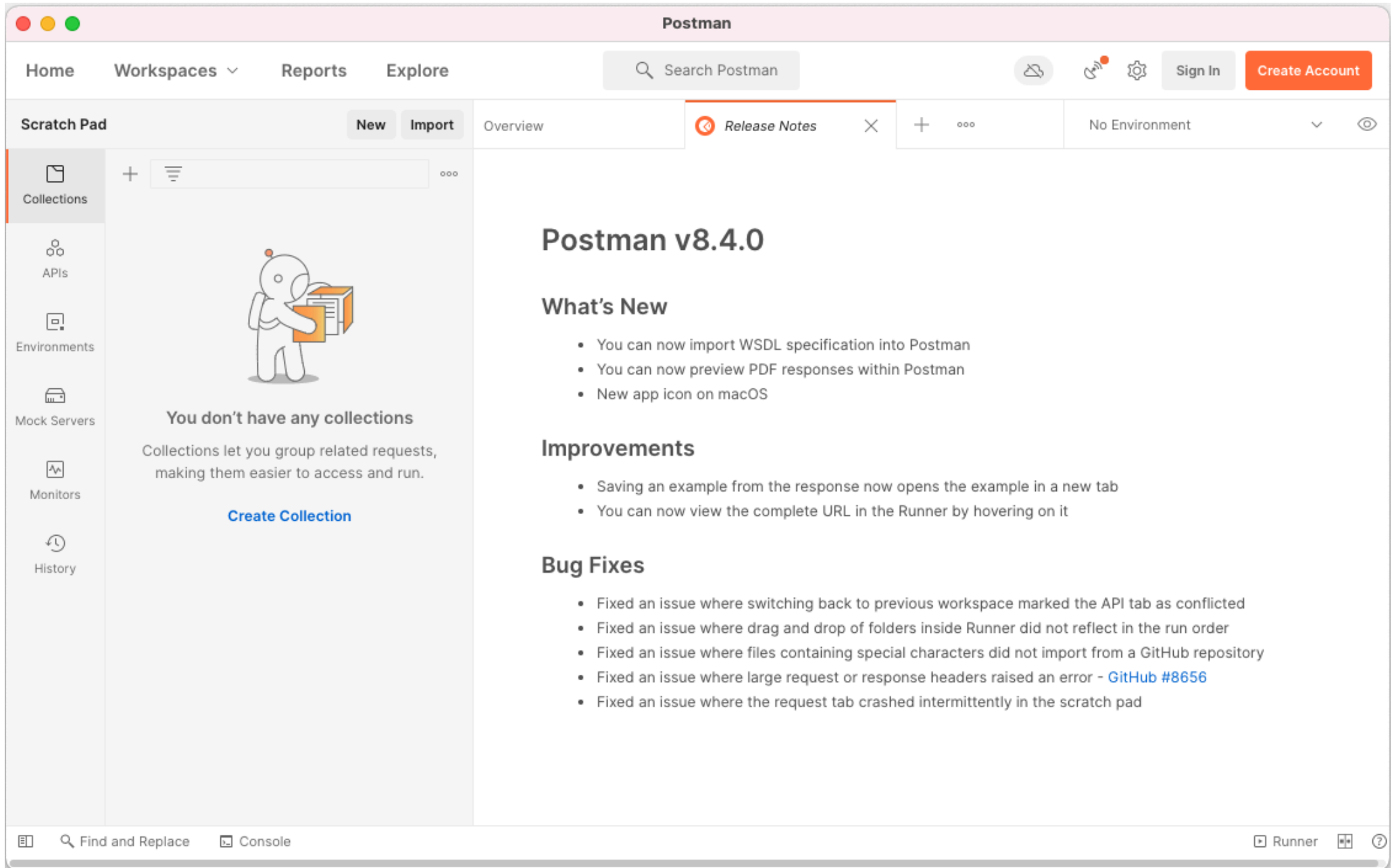
Luckily for us, there are some excellent tools out there to make interacting with APIs, and customizing all different parts of the request (the headers, HTTP verb, URL, and body) easier for us!

Using Postman as an API Client

The tool we'll be demonstrating in this lesson is [Postman](https://postman.com)  [_\(https://postman.com\)_](https://postman.com). Postman is an API Client tool that will help with testing and development of our API.

First, head to this page and download Postman: <https://www.postman.com/downloads/>  [_\(https://www.postman.com/downloads/\)_](https://www.postman.com/downloads/). Make sure to download the app rather than use the web version. If you have any issues downloading, make sure to check out the [Postman docs](https://learning.postman.com/docs/getting-started/installation-and-updates/)  [_\(https://learning.postman.com/docs/getting-started/installation-and-updates/\)_](https://learning.postman.com/docs/getting-started/installation-and-updates/) for help.

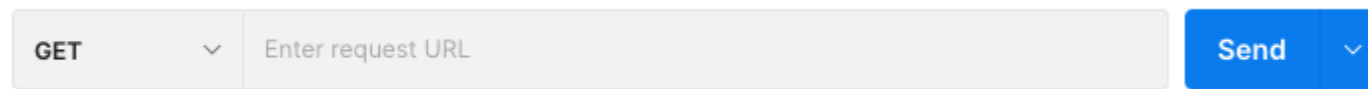
Once you've downloaded Postman and signed in, you should see a screen like this:



To show what we can do with Postman, we're going to be using the [JSON Placeholder API](https://jsonplaceholder.typicode.com/guide/)  (<https://jsonplaceholder.typicode.com/guide/>), a free resource where we can try out different requests. For our first request, we'll make a simple GET request to

<https://jsonplaceholder.typicode.com/posts/1> .

Click on the plus sign to open a new tab. You should see a field where you can enter your request:

The image shows a simplified version of the Postman interface. It consists of a light gray rectangular box. On the left side of this box, there is a smaller gray box containing the text 'GET' and a small downward-pointing chevron icon. To the right of this is a larger text input field with the placeholder text 'Enter request URL'. To the right of the input field is a blue button with the text 'Send' and a small downward-pointing chevron icon.

Enter the URL in the field and click send:

Release Notes

GET https://jsonplaceh...

+

...

No Environment

https://jsonplaceholder.typicode.com/posts/1

Save

GET

https://jsonplaceholder.typicode.com/posts/1

Send

Params

Auth

Headers (8)

Body

Pre-req.

Tests

Settings

Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
	Key	Value	Description		

Body

Cookies (1)

Headers (27)

Test Results

200 OK124 ms1.53 KB

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

{

2

"userId": 1,

3

"id": 1,

4

"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",

5

"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit

https://moringa.instructure.com/courses/1054/pages/testing-apis-with-postman?module_item_id=177773

6/11

molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"

6

}

You'll see we receive a JSON object as a response!

This API also lets us make non-GET requests, so let's see how we can use Postman for making a POST request. We'll try recreating this `fetch` request in Postman:

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    title: "foo",
    body: "bar",
    userId: 1,
  }),
});
```

First, update the **URL** to be `https://jsonplaceholder.typicode.com/posts`. Then, click the dropdown next to the URL bar, and change the **HTTP verb** from a GET request to a POST request.

We can also use Postman to customize what **headers** are being sent. Headers are additional metadata that we can send with requests. In the `fetch` example above, the `Content-Type` header tells the server that we expect to receive `application/json` in the response. To add a header, click the 'Headers' tab, and add a new key of `Content-Type` with a value of `application/json`.

Finally, we'll need some way of sending a JSON formatted string in the **body** of the request. Click the 'Body' tab in Postman, then click the dropdown and select 'raw' to specify that we are sending a raw string of data. Then, select JSON from the next dropdown to specify that the

string will be formatted as JSON. Now we can add a JSON formatted string to the text area below the dropdown:

```
{  
  "title": "foo",  
  "body": "bar",  
  "userId": 1  
}
```

Note: for the string to be considered valid JSON, all the keys on the object must be in **double-quotes**. There also can't be any trailing commas after the last key-value pair. You can use [this JSON tool](https://jsonlint.com/)  [\(https://jsonlint.com/\)](https://jsonlint.com/) to validate your JSON if you need to troubleshoot.

With all that in place, click send:

Release Notes

POST https://jsonplace...

No Environment

https://jsonplaceholder.typicode.com/posts

Save

Send

Params Auth Headers (11) **Body** Pre-req. Tests Settings Cookies

raw JSON Beautify

```
1 {
2   "title": "foo",
3   "body": "bar",
4   "userId": 1
5 }
```

Body **Cookies (1)** Headers (26) Test Results 201 Created 1605 ms 1.23 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "title": "foo",
3   "body": "bar",
4   "userId": 1,
5   "id": 101
}
```

Success! The API has received our request and sent back a response representing a newly created **post** :

```
{
  "title": "foo",
  "body": "bar",
  "userId": 1,
  "id": 101
}
```

You'll also notice some additional info comes back in the response, such as a status code of **201 Created** that indicates the successful creation of a resource.

Experiment with Postman by making requests to other endpoints for the [JSON Placeholder API](https://jsonplaceholder.typicode.com/guide/)  (<https://jsonplaceholder.typicode.com/guide/>). Try making a PATCH request and a DELETE request. What changes between these different requests? What does this API send as a response if you make a bad request?

Conclusion

API clients like Postman are extremely valuable to have in your API developer toolkit. They make it simple to interact with servers by giving you full control over how the response is being sent, with a nice interface for customizing the HTTP verb, URL, headers, and body.

In future lessons, we'll be expanding our API to handle non-GET requests, so being able to use a tool like Postman will make our API development much easier! It's also a great tool to use if you're exploring a third-party API for use in your projects.

Resources

- [Postman Documentation](https://learning.postman.com/docs/getting-started/installation-and-updates/)  [\(https://learning.postman.com/docs/getting-started/installation-and-updates/\)](https://learning.postman.com/docs/getting-started/installation-and-updates/)